Końcowa dokumentacja projektu

System wyznaczania współrzędnych dla wizualizacji grafów planarnych

Implementacja w języku C

Autorzy:
Krzysztof Wasilewski, Jakub Pietrzkiewicz

Data: 20.04.2026

Spis treści

1 Wprowadzenie ... 4
1.1 Cel projektu ... 4
2 Charakterystyka funkcjonalna aplikacji ... 5
2.1 Funkcje udostępniane użytkownikowi ... 5
2.2 Interfejs uruchomieniowy CLI ... 5
2.3 Przykładowe scenariusze użycia ... 5
3 Architektura oprogramowania ... 6
3.1 Podział na moduły ... 6
3.2 Przebieg działania programu ... 6
4 Model danych ... 7
4.1 Struktury podstawowe ... 7
4.2 Struktury pomocnicze ... 7
4.3 Kody statusu ... 8
5 Specyfikacja wejścia i wyjścia ... 9
5.1 Wejście tekstowe ... 9
5.2 Mechanizm dwuprzebiegowy odczytu ... 9
5.3 Wyjście tekstowe ... 9
5.4 Wyjście binarne ... 9
6 Szczegółowy opis kodu ... 10
6.1 Moduł `main` ... 10
6.2 Moduł `graph` ... 12
6.3 Moduł `fruchterman` ... 14
6.4 Moduł `adjacency_list` ... 16
6.5 Moduł `tutte` ... 17
6.6 Moduł `utils` ... 21
7 Szczegółowa analiza funkcji ... 23
7.1 `main(int argc, char *argv[])` ... 23
7.2 `load_graph(char path[], Graph **graph_out)` ... 23
7.3 `save_graph_as_text(Graph *graph, char path[])` ... 23
7.4 `save_graph_as_binary(Graph *graph, char path[])` ... 24
7.5 `run_fruchterman(...)` ... 24
7.6 `create_adjacency_list(...)` i `free_adjacency_list(...)` ... 24
7.7 `run_tutte(...)` ... 24
7.8 Funkcje pomocnicze `utils` ... 25
8 Opis algorytmiczny ... 26
8.1 Fruchterman-Reingold ... 26
8.2 Tutte-like embedding ... 26
9 Zarządzanie pamięcią i odporność ... 28
9.1 Strategia alokacji ... 28
9.2 Strategia cleanup ... 28
9.3 Potencjalne punkty ryzyka pamięci ... 28
10 Obsługa błędów ... 29
10.1 Kategorie błędów ... 29
10.2 Kody powrotu i znaczenie ... 29
10.3 Komunikaty diagnostyczne ... 29
11 Proces budowania i uruchamiania ... 30
11.1 Makefile ... 30
11.2 Instrukcja uruchomienia ... 31
12 Testowanie i weryfikacja ... 32

12.1 Zakres testów jednostkowych ... 32
12.2 Wynik aktualnego uruchomienia ... 34
12.3 Ograniczenia algorytmiczne ... 34
12.4 Ograniczenia implementacyjne ... 34
12.5 Ryzyka eksploatacyjne ... 34
13 Podsumowanie ... 35
14 Załącznik A: Skrócona referencja API (C) ... 36
14.1 graph.h ... 36
14.2 fruchterman.h ... 36
14.3 tutte.h ... 36
14.4 adjacency_list.h ... 36
14.5 utils.h ... 36
15 Załącznik B: Przykładowe dane ... 37
15.1 Przykładowy plik wejściowy ... 37
15.2 Przykładowe uruchomienie ... 37
15.3 Przykładowy wynik tekstowy ... 37

1 Wprowadzenie

Projekt realizuje aplikację konsolową w języku C, która dla grafu planarnego zapisanego jako lista krawędzi wyznacza współrzędne wierzchołków przeznaczone do dalszej wizualizacji. Program implementuje dwa algorytmy:

- Fruchterman-Reingold (algorytm siłowy),
- Tutte embedding (osadzenie z relaksacją sąsiadów i ustalonym brzegiem).

Aplikacja obsługuje uruchamianie parametryzowane z linii poleceń, wejście tekstowe oraz dwa formaty wyjścia (tekstowy i binarny). Kod ma budowę modularną, pokrycie testami jednostkowymi i jednolity system kodów powrotu. Dokument przedstawia:

- szczegółową analizę kodu,
- opis architektury i przepływu sterowania,
- formaty danych,
- ocenę zgodności z wymaganiami,
- ograniczenia i propozycje rozwoju.

1.1 Cel projektu

Celem projektu jest dostarczenie narzędzia CLI, które przyjmuje graf opisany jako lista krawędzi i generuje współrzędne 2D wierzchołków, aby możliwa była czytelna i estetyczna wizualizacja struktury grafu. Kluczowe założenia:

- prosty i przewidywalny interfejs uruchomieniowy,
- możliwość wyboru algorytmu rozmieszczenia,
- eksport wyników do formatu tekstowego lub binarnego,
- przenośna i testowalna implementacja C.

2 Charakterystyka funkcjonalna aplikacji

2.1 Funkcje udostępniane użytkownikowi

Program udostępnia funkcje:

- odczyt grafu z pliku tekstowego,
- uruchomienie wybranego algorytmu layoutu,
- konfigurację liczby iteracji oraz parametrów przestrzeni roboczej,
- zapis wyniku do formatu tekstowego,
- zapis wyniku do formatu binarnego.

2.2 Interfejs uruchomieniowy CLI

Składnia ogólna:

```
./graph [opcje] <input_file> <output_file>
```

Opcje:

- `-a`, `--algorithm <fruchterman|tutte>`
- `-i`, `--iterations <int>`
- `-t`, `--temperature <double>`
- `-s`, `--size <double>`
- `-f`, `--format <text|binary>`
- `-h`, `--help`

Wartości domyślne:

- `algorithm = fruchterman`
- `iterations = 1000`
- `temperature = 10.0`
- `size = 1000.0`
- `format = text`

2.3 Przykładowe scenariusze użycia

Domyślne uruchomienie:

```
./graph dane.txt wynik.txt
```

Uruchomienie algorytmu Tutte:

```
./graph -a tutte dane.txt wynik.txt
```

Eksport binarny:

```
./graph -f binary dane.txt wynik.bin
```

Uruchomienie z jawną konfiguracją:

```
./graph -a fruchterman -i 2500 -t 12.5 -s 1400 -f text dane.txt wynik.txt
```

3 Architektura oprogramowania

3.1 Podział na moduły

Kod źródłowy jest podzielony na 6 modułów:

- `main` - punkt wejściowy programu i orkiestracja przepływu,
- `graph` - odczyt i zapis grafu, mapowanie identyfikatorów,
- `fruchterman` - algorytm siłowy,
- `tutte` - algorytm osadzenia Tutte,
- `adjacency_list` - pomocnicza reprezentacja sąsiadów,
- `utils` - wspólne kody statusu i struktura listy jednokierunkowej.

3.2 Przebieg działania programu

Przebieg pojedynczego uruchomienia:

1. `main` parsuje argumenty i waliduje parametry.
2. `load_graph` odczytuje plik i buduje strukturę `Graph`.
3. `main` wybiera algorytm (`run_fruchterman` albo `run_tutte`).
4. Wynik jest zapisywany przez `save_graph_as_text` lub `save_graph_as_binary`.
5. Pamięć jest zwalniana przez `free_graph`.
6. Program zwraca kod statusu.

4 Model danych

4.1 Struktury podstawowe

Model grafu oparty jest o tablice dynamiczne:

Implementacja w kodzie znajduje się w module `graph`:

```
typedef struct {
  int id;
  double x, y;
} Node;

typedef struct {
  int first_node_index;
  int second_node_index;
  double weight;
  char label[33];
} Edge;

typedef struct {
  Node *nodes;
  int nodes_count;
  Edge *edges;
  int edges_count;
} Graph;
```

Znaczenie pól:

- `Node`:
  - `id` - identyfikator logiczny,
  - `x`, `y` - współrzędne.
- `Edge`:
  - `first_node_index`, `second_node_index` - indeksy do tablicy `nodes`,
  - `weight` - waga krawędzi,
  - `label[33]` - etykieta (do 32 znaków + terminator).
- `Graph`:
  - `nodes`, `nodes_count`,
  - `edges`, `edges_count`.

4.2 Struktury pomocnicze

W module `adjacency_list`:

```
typedef struct Neighbor {
  int node_index;
  double weight;
  struct Neighbor *next;
} Neighbor;

typedef struct {
  Neighbor **adjacency_list;
  int nodes_count;
  int *degrees;
} AdjacencyList;
```

- `Neighbor` - element listy sąsiadów (`node_index`, `weight`, `next`),

- `AdjacencyList` - tablica list sąsiadów i tablica stopni wierzchołków.

W module `utils`:

```
typedef struct List {
  int data;
  struct List *next;
} List;
```

- `List` - prosta lista jednokierunkowa używana przy pierwszym skanie pliku do budowy zbioru unikalnych ID.

4.3 Kody statusu

System kodów powrotu (enum):

- `SUCCESS = 0`
- `ARGUMENTS_ERROR = 1`
- `FILE_ERROR = 2`
- `INPUT_FORMAT_ERROR = 3`
- `ALGORITHM_ERROR = 4`
- `MEMORY_ERROR = 5`

5 Specyfikacja wejścia i wyjścia

5.1 Wejście tekstowe

Kazda linia danych ma postac:

```
<label> <nodeA> <nodeB> <weight>
```

Przykład:

```
AB 1 2 1.0
BC 2 3 0.8
CA 3 1 1.2
```

Zasady parsowania:

- parser akceptuje białe znaki (`space`, `tab`, `CR`, `LF`),
- parser ignoruje całe linie komentarza zaczynające się od `#`,
- parser toleruje komentarze inline po rekordzie,
- etykieta jest ograniczona do 32 znakow (`%32s`).

5.2 Mechanizm dwuprzebiegowy odczytu

Odczyt grafu realizowany jest dwuprzebiegowo:

1. Przebieg pierwszy:
   - zliczenie krawędzi,
   - zbudowanie zbioru unikalnych identyfikatorów wierzchołków.
2. Przebieg drugi:
   - mapowanie identyfikatorów na indeksy,
   - wypełnienie tablicy `edges`.

Korzyść: ograniczenie reallocacji i precyzyjna alokacja pamięci.

5.3 Wyjście tekstowe

Format:

```
<node_id> <x> <y>
```

Przykład:

```
1 0.000000 0.000000
2 1000.000000 0.000000
3 500.000000 866.000000
```

5.4 Wyjście binarne

Dla każdego wierzchołka zapisywany jest rekord 20-bajtowy:

- 4 bajty: `int id`,
- 8 bajtów: `double x`,
- 8 bajtów: `double y`.

Uwagi integracyjne:

- plik nie zawiera nagłówka,
- kolejność rekordów odpowiada kolejności w tablicy `nodes` (po sortowaniu po `id`),

6 Szczegółowy opis kodu

6.1 Moduł `main`

Rola:

- inicjalizacja parametrów domyślnych,
- parsowanie opcji CLI,
- walidacja danych sterujących,
- wybór algorytmu i formatu zapisu,
- jednolita obsługa kodów wyjścia.

Najważniejsze decyzje implementacyjne:

- wymaganie obecności dwóch argumentów pozycyjnych (`input_file`, `output_file`),
- walidacja `iterations > 0`, `temperature > 0`, `size > 0`,
- jawna walidacja nazw algorytmu i formatu.

Opis parsera argumentów:

- parser przechodzi po `argv` liniowo,
- opcje wymagające parametru zużywają kolejny token,
- `-h/--help` kończy program kodem sukcesu,
- nieznana wartość wejścia skutkuje `ARGUMENTS_ERROR`.

Fragment pokazujący inicjalizację parametrów i parsowanie CLI:

```c
int main(int argc, char *argv[]) {
  // Domyślne wartości parametrów
  char *input_file = NULL;
  char *output_file = NULL;
  char *algorithm = "fruchterman";
  char *format = "text";
  int iterations = 1000;
  double temperature = 10.0;
  double size = 1000.0;

  // Parsowanie argumentów wiersza poleceń
  for (int i = 1; i < argc; i++) {
    if (strcmp(argv[i], "-a") == 0 || strcmp(argv[i], "--algorithm") == 0) {
      if (++i < argc)
        algorithm = argv[i];
    } else if (strcmp(argv[i], "-i") == 0 ||
            strcmp(argv[i], "--iterations") == 0) {
      if (++i < argc)
        iterations = atoi(argv[i]);
    } else if (strcmp(argv[i], "-t") == 0 ||
            strcmp(argv[i], "--temperature") == 0) {
      if (++i < argc)
        temperature = atof(argv[i]);
    } else if (strcmp(argv[i], "-s") == 0 || strcmp(argv[i], "--size") == 0) {
      if (++i < argc)
        size = atof(argv[i]);
    } else if (strcmp(argv[i], "-f") == 0 || strcmp(argv[i], "--format") == 0) {
      if (++i < argc)
        format = argv[i];
    } else if (strcmp(argv[i], "-h") == 0 || strcmp(argv[i], "--help") == 0) {
      printf(
          "=== Graph Layout Tool ===\n"
```

```
          "Użycie programu: ./graph [opcje] <input_file> <output_file>\n\n"
          "Opcje:\n"
          "  -a, --algorithm <algorytm>    Wybór algorytmu (fruchterman, "
          "tutte)\n"
          "  -i, --iterations <liczba>     Liczba iteracji (domyślnie: 1000)\n"
          "  -t, --temperature <liczba>    Początkowa temperatura (domyślnie: "
          "10.0)\n"
          "  -s, --size <liczba>           Rozmiar obszaru layoutu (domyślnie: "
          "1000.0)\n"
          "  -f, --format <format>         Format wyjściowy (text, binary)\n"
          "  -h, --help                    Wyświetlenie tej pomocy\n");
      return SUCCESS;
    } else if (input_file == NULL && i == argc - 2) {
      input_file = argv[i];
    } else if (output_file == NULL && i == argc - 1) {
      output_file = argv[i];
    } else {
      fprintf(stderr, "Error: Unknown option '%s'\n", argv[i]);
      return ARGUMENTS_ERROR;
    }
  }
}
```

Fragment pokazujący obsługę pomocy i walidację argumentów:

```
  if (input_file == NULL || output_file == NULL) {
    fprintf(stderr, "Error: Input and output files must be specified.\n");
    return ARGUMENTS_ERROR;
  }
  if (iterations <= 0) {
    fprintf(stderr, "Error: Invalid iteration value\n");
    return ARGUMENTS_ERROR;
  }
  if (temperature <= 0) {
    fprintf(stderr, "Error: Invalid temperature value\n");
    return ARGUMENTS_ERROR;
  }
  if (size <= 0) {
    fprintf(stderr, "Error: Invalid size value\n");
    return ARGUMENTS_ERROR;
  }
```

Fragment pokazujący końcowy wybór formatu i zwalnianie pamięci:

```
  Graph *graph = NULL;
  int status = load_graph(input_file, &graph);
  if (status != SUCCESS) {
    return status;
  }

  // Uruchomienie wybranego algorytmu layoutu grafu
  if (strcmp(algorithm, "fruchterman") == 0) {
    status = run_fruchterman(graph, temperature, iterations, size);
  } else if (strcmp(algorithm, "tutte") == 0) {
    status = run_tutte(graph, size, iterations);
  } else {
    fprintf(stderr,
        "Error: Unknown algorithm '%s'. Use 'fruchterman' or 'tutte'.\n",
        algorithm);
    free_graph(graph);
```

```
    return ARGUMENTS_ERROR;
  }
  if (status != SUCCESS) {
    free_graph(graph);
    return status;
  }

  if (strcmp(format, "text") == 0) {
    status = save_graph_as_text(graph, output_file);
  } else if (strcmp(format, "binary") == 0) {
    status = save_graph_as_binary(graph, output_file);
  } else {
    fprintf(stderr, "Error: Unknown format '%s'. Use 'text' or 'binary'.\n",
        format);
    free_graph(graph);
    return ARGUMENTS_ERROR;
  }
  free_graph(graph);
  return status;
}
```

6.2 Moduł `graph`

Rola:

- odczyt grafu z pliku,
- mapowanie ID wierzchołków na indeksy tablicowe,
- zapis wyników,
- zwalnianie pamięci struktury `Graph`.

Szczegóły implementacyjne:

- dane wierzchołków są sortowane funkcją `qsort`,
- mapowanie ID realizowane przez `bsearch` (`get_node_index`),
- podczas odczytu używane jest pomocnicze pomijanie komentarzy i białych znaków.

Mocne strony:

- dobra wydajność mapowania ID (wyszukiwanie binarne),
- jasny podział na odczyt i zapis,
- ograniczenie rozmiaru etykiety zabezpiecza bufor.

Krótki fragment pokazujący dwuprzebiegowy odczyt i mapowanie identyfikatorów:

```
int load_graph(char path[], Graph **graph_out) {
  FILE *file = fopen(path, "r");
  if (file == NULL) {
    fprintf(stderr, "Error: Cannot open input file: %s\n", path);
    return FILE_ERROR;
  }

  char label[33];
  int first_node, second_node;
  double weight;
  int edges_count = 0;
  int nodes_count = 0;
  List *unique_nodes = NULL;
  int line_number = 1;
  char c;
```

```c
  // Pierwsze przejście: zliczanie krawędzi i identyfikacja unikalnych wierzchołków
  while ((c = fgetc(file)) != EOF) {
    skip_whitespace_and_comments(file, &c, &line_number);
    if (c == EOF)
      break;

    ungetc(c, file);
    int result = fscanf(file, "%32s %d %d %lf", label, &first_node,
               &second_node, &weight);

    if (result == 4) {
      edges_count++;
      int ids[2] = {first_node, second_node};
      for (int i = 0; i < 2; i++) {
        if (!list_contains(unique_nodes, ids[i])) {
          list_prepend(&unique_nodes, ids[i]);
          nodes_count++;
        }
      }
    } else if (result != EOF) {
      list_free(unique_nodes);
      fclose(file);
      return INPUT_FORMAT_ERROR;
    }
  }

  Graph *graph = (Graph *)malloc(sizeof(Graph));
  if (graph == NULL) {
    list_free(unique_nodes);
    fclose(file);
    return MEMORY_ERROR;
  }

  graph->nodes_count = nodes_count;
  graph->edges_count = edges_count;
  graph->nodes = (Node *)malloc(nodes_count * sizeof(Node));
  graph->edges = (Edge *)malloc(edges_count * sizeof(Edge));

  int node_index = 0;
  for (List *current = unique_nodes; current != NULL; current = current->next) {
    graph->nodes[node_index].id = current->data;
    graph->nodes[node_index].x = 0.0;
    graph->nodes[node_index].y = 0.0;
    node_index++;
  }
  list_free(unique_nodes);
  qsort(graph->nodes, graph->nodes_count, sizeof(Node), compare_nodes_by_id);
}
```

Fragment pokazujący zapis tekstowy i binarny oraz wyszukiwanie indeksu:

```c
int save_graph_as_text(Graph *graph, char path[]) {
  FILE *file = fopen(path, "w");
  if (file == NULL)
    return FILE_ERROR;
  for (int i = 0; i < graph->nodes_count; i++) {
    fprintf(file, "%d %lf %lf\n", graph->nodes[i].id, graph->nodes[i].x,
            graph->nodes[i].y);
  }
  fclose(file);
```

```
  return SUCCESS;
}

int get_node_index(Graph *graph, int node_id) {
  Node key = {.id = node_id};
  Node *result = (Node *)bsearch(&key, graph->nodes, graph->nodes_count,
                                 sizeof(Node), compare_nodes_by_id);
  return (result == NULL) ? -1 : (int)(result - graph->nodes);
}
```

Fragment pokazujący zwalnianie zasobów i pomijanie komentarzy:

```
int free_graph(Graph *graph) {
  if (graph == NULL)
    return SUCCESS;
  if (graph->nodes)
    free(graph->nodes);
  if (graph->edges)
    free(graph->edges);
  free(graph);
  return SUCCESS;
}

static void skip_whitespace_and_comments(FILE *file, char *c,
                                         int *line_number) {
  while (*c != EOF) {
    if (*c == '#') {
      while ((*c = fgetc(file)) != EOF && *c != '\n')
        ;
    } else {
      break;
    }
  }
}
```

Fragment pokazujący drugi przebieg i mapowanie na indeksy:

```
  rewind(file);
  int edge_index = 0;
  line_number = 1;
  while ((c = fgetc(file)) != EOF) {
    skip_whitespace_and_comments(file, &c, &line_number);
    if (c == EOF)
      break;

    ungetc(c, file);
    if (fscanf(file, "%32s %d %d %lf", label, &first_node, &second_node,
               &weight) == 4) {
      graph->edges[edge_index].first_node_index =
          get_node_index(graph, first_node);
      graph->edges[edge_index].second_node_index =
          get_node_index(graph, second_node);
    }
  }
```

6.3 Moduł `fruchterman`

Rola:

- implementacja iteracyjnego algorytmu siłowego.

Fazy obliczeń:

1. Losowa inicjalizacja pozycji w kwadracie `[0,size] x [0,size]`.
2. Obliczenie sił odpychania dla każdej pary wierzchołków.
3. Obliczenie sił przyciągania dla każdej krawędzi.
4. Aktualizacja pozycji z ograniczeniem kroku przez temperature.
5. Chłodzenie liniowe temperatury.

Stabilizacja numeryczna:

- w odległości dodawane jest `1e-9`, aby uniknąć dzielenia przez zero.

Uwagi praktyczne:

- wynik nie jest deterministyczny przez `srand(time(NULL))`,
- dla grafów bez krawędzi funkcja zwraca `ARGUMENTS_ERROR`.

Krótki fragment pokazujący obliczanie sił i aktualizację pozycji:

```
for (int iter = 0; iter < max_iterations; iter++) {
  for (int i = 0; i < liczba_w; i++) {
    sila_x[i] = 0.0;
    sila_y[i] = 0.0;
  }
  // Siły odpychania
  for (int a = 0; a < liczba_w; a++) {
    for (int b = 0; b < liczba_w; b++) {
      if (a != b) {
        double dx = graph->nodes[a].x - graph->nodes[b].x;
        double dy = graph->nodes[a].y - graph->nodes[b].y;
        double odleglosc = sqrt(dx * dx + dy * dy) + 1e-9;
        double fr = (k * k) / odleglosc;
        sila_x[a] += (dx / odleglosc) * fr;
        sila_y[a] += (dy / odleglosc) * fr;
      }
    }
  }
  // Siły przyciągania
  for (int e = 0; e < liczba_k; e++) {
    int a = graph->edges[e].first_node_index;
    int b = graph->edges[e].second_node_index;
    double waga = graph->edges[e].weight;
    double dx = graph->nodes[a].x - graph->nodes[b].x;
    double dy = graph->nodes[a].y - graph->nodes[b].y;
    double odleglosc = sqrt(dx * dx + dy * dy) + 1e-9;
    double fa = waga * (odleglosc * odleglosc) / k;
    sila_x[a] -= (dx / odleglosc) * fa;
    sila_y[a] -= (dy / odleglosc) * fa;
    sila_x[b] += (dx / odleglosc) * fa;
    sila_y[b] += (dy / odleglosc) * fa;
  }
}
```

Fragment pokazujący inicjalizację i przyciąganie krawędzi:

```
  double area = size * size;
  // k - optymalna odległość między wierzchołkami
  double k = sqrt(area / liczba_w);
  double temperature = initial_temperature;
  srand(time(NULL));
```

```c
// Losowa inicjalizacja pozycji wierzchołków
for (int i = 0; i < liczba_w; i++) {
  graph->nodes[i].x = random_double(0.0, size);
  graph->nodes[i].y = random_double(0.0, size);
}

double *sila_x = (double *)calloc(liczba_w, sizeof(double));
double *sila_y = (double *)calloc(liczba_w, sizeof(double));
if (sila_x == NULL || sila_y == NULL) {
  fprintf(stderr, "Error: Memory allocation failed.\n");
  free(sila_x);
  free(sila_y);
  return MEMORY_ERROR;
}
```

Fragment pokazujący ograniczanie do obszaru i chłodzenie:

```c
for (int i = 0; i < liczba_w; i++) {
  double wek_sily = sqrt(sila_x[i] * sila_x[i] + sila_y[i] * sila_y[i]);
  if (wek_sily > 0) {
    double przesuniecie = fmin(wek_sily, temperature);
    graph->nodes[i].x += (sila_x[i] / wek_sily) * przesuniecie;
    graph->nodes[i].y += (sila_y[i] / wek_sily) * przesuniecie;
  }
  // Ograniczenie do obszaru
  if (graph->nodes[i].x < 0.0)
    graph->nodes[i].x = 0.0;
  if (graph->nodes[i].x > size)
    graph->nodes[i].x = size;
  if (graph->nodes[i].y < 0.0)
    graph->nodes[i].y = 0.0;
  if (graph->nodes[i].y > size)
    graph->nodes[i].y = size;
}
// Chłodzenie
temperature -= initial_temperature / max_iterations;
if (temperature < 0.0) {
  temperature = 0.0;
}
free(sila_x);
free(sila_y);
return SUCCESS;
```

6.4 Moduł `adjacency_list`

Rola:

- konwersja tablicy krawędzi grafu do listy sąsiedztwa,
- policzenie stopni wierzchołków,
- zwolnienie pamięci pomocniczej.

Szczegóły:

- każda krawędź dodawana dwukierunkowo (graf nieskierowany),
- `degrees[i]` jest inkrementowane dla obu końców krawędzi,
- alokacja przez `calloc` zeruje wskaźniki i liczniki.

Krótki fragment pokazujący budowę listy sąsiedztwa:

```c
int create_adjacency_list(Graph *graph, AdjacencyList **adj_list_out) {
  adj_list->adjacency_list =
```

```
      (Neighbor **)calloc(graph->nodes_count, sizeof(Neighbor *));
  adj_list->degrees = (int *)calloc(graph->nodes_count, sizeof(int));

  for (int i = 0; i < graph->edges_count; i++) {
    int first_index = graph->edges[i].first_node_index;
    int second_index = graph->edges[i].second_node_index;
    double weight = graph->edges[i].weight;

    add_neighbor(&adj_list->adjacency_list[first_index], second_index, weight);
    add_neighbor(&adj_list->adjacency_list[second_index], first_index, weight);
  }
}
```

Fragment pokazujący zwalnianie list sąsiedztwa i dodawanie sąsiada:

```
void free_adjacency_list(AdjacencyList *adj_list) {
  if (adj_list == NULL)
    return;

  if (adj_list->adjacency_list != NULL) {
    for (int i = 0; i < adj_list->nodes_count; i++) {
      Neighbor *current = adj_list->adjacency_list[i];
      while (current != NULL) {
        Neighbor *next = current->next;
        free(current);
        current = next;
      }
    }
  }
  free(adj_list->degrees);
  free(adj_list);
}

static int add_neighbor(Neighbor **list, int node_index, double weight) {
  Neighbor *neighbor = (Neighbor *)malloc(sizeof(Neighbor));
  neighbor->node_index = node_index;
  neighbor->weight = weight;
  neighbor->next = *list;
  *list = neighbor;
  return SUCCESS;
}
```

Fragment pokazujący strukturę sąsiada i pola listy:

```
typedef struct Neighbor {
  int node_index;
  double weight;
  struct Neighbor *next;
} Neighbor;

typedef struct {
  Neighbor **adjacency_list;
  int nodes_count;
  int *degrees;
} AdjacencyList;
```

6.5 Moduł `tutte`

Rola:

- implementacja osadzenia Tutte z automatycznym wyznaczeniem ramy.

Przebieg:

1. Dla `nodes_count < 4` uruchamiana jest procedura specjalna.
2. Dla większych grafów tworzona jest lista sąsiedztwa.
3. Proba znalezienia cyklu brzegowego (`find_cycle_perimeter`).
4. Jeśli brak sukcesu, fallback do 4 wierzchołków o najwyższych stopniach (`find_cycle_backup`).
5. Rozmieszczenie ramy na obwodzie kwadratu.
6. Inicjalizacja wewnętrznych wierzchołków w centrum.
7. Iteracyjna relaksacja do zbieżności (`epsilon = 0.0001`) lub limitu iteracji.

Krótki fragment pokazujący wyznaczanie cyklu i relaksację wierzchołków:

```
int run_tutte(Graph *graph, double size, int max_iterations) {
  AdjacencyList *adj_list;
  int *cycle = NULL;
  int cycle_len = 0;

  status = create_adjacency_list(graph, &adj_list);
  status = find_cycle(adj_list, &cycle, &cycle_len);
  place_cycle_on_square(graph, cycle, cycle_len, size);

  for (int iter = 0; iter < max_iterations; iter++) {
    for (int i = 0; i < graph->nodes_count; i++) {
      if (!is_cycle[i]) {
        graph->nodes[i].x = sum_weighted_x / sum_weights;
        graph->nodes[i].y = sum_weighted_y / sum_weights;
      }
    }
  }
}
```

Fragment pokazujący heurystykę szukania cyklu i fallback:

```
static int find_cycle(AdjacencyList *adj_list, int **cycle, int *cycle_len) {
  return (find_cycle_perimeter(adj_list, cycle, cycle_len) == SUCCESS)
             ? SUCCESS
             : find_cycle_backup(adj_list, cycle, cycle_len);
}

static int find_cycle_backup(AdjacencyList *adj_list, int **cycle_out,
                             int *cycle_len_out) {
  int *cycle = (int *)malloc(4 * sizeof(int));
  int max_indices[4] = {0, 1, 2, 3};
  *cycle_out = cycle;
  *cycle_len_out = 4;
  return SUCCESS;
}
```

Fragment pokazujący rozmieszczenie na obwodzie i relaksację środka:

```
static void place_cycle_on_square(Graph *graph, int *cycle, int cycle_len,
                                  double size) {
  double perimeter = 4.0 * size;
  double step = perimeter / cycle_len;
  for (int i = 0; i < cycle_len; i++) {
    double pos = i * step;
    double x, y;
    graph->nodes[cycle[i]].x = x;
    graph->nodes[cycle[i]].y = y;
  }
```

```
}

static void handle_graph_below_four_nodes(Graph *graph, double size) {
  double coords[][2] = {{size / 2.0, size / 2.0}, {0.0, 0.0}};
}
```

Fragment pokazujący początek funkcji `run_tutte`:

```
static int count_common_neighbors(int u, int v, AdjacencyList *adj);
static int find_cycle_perimeter(AdjacencyList *adj_list, int **cycle_out,
                                int *cycle_len_out);
static int find_cycle_backup(AdjacencyList *adj_list, int **cycle_out,
                             int *cycle_len_out);
static int find_cycle(AdjacencyList *adj_list, int **cycle, int *cycle_len);
static void place_cycle_on_square(Graph *graph, int *cycle, int cycle_len,
                                  double size);
static void handle_graph_below_four_nodes(Graph *graph, double size);

int run_tutte(Graph *graph, double size, int max_iterations) {
  int status;
  if (graph->nodes_count < 4) {
    handle_graph_below_four_nodes(graph, size);
    return SUCCESS;
  }

  AdjacencyList *adj_list;
  status = create_adjacency_list(graph, &adj_list);
  if (status != SUCCESS) {
    fprintf(stderr, "Error: Mamory allocation failed\n");
    return status;
  }
```

Fragment pokazujący wyznaczanie cyklu i start relaksacji:

```
  int *cycle = NULL;
  int cycle_len = 0;

  // Znajdowanie cyklu brzegowego (zewnętrzna ściana grafu planarnego)
  status = find_cycle(adj_list, &cycle, &cycle_len);
  if (status != SUCCESS) {
    free_adjacency_list(adj_list);
    return status;
  }

  // Umieszczenie wierzchołków cyklu na brzegu kwadratu
  place_cycle_on_square(graph, cycle, cycle_len, size);

  bool *is_cycle = (bool *)calloc(graph->nodes_count, sizeof(bool));
  if (is_cycle == NULL) {
    free(cycle);
    free_adjacency_list(adj_list);
    fprintf(stderr, "Error: Memory allocation failed\n");
    return MEMORY_ERROR;
  }

  for (int i = 0; i < cycle_len; i++) {
    is_cycle[cycle[i]] = true;
  }
```

Fragment pokazujący heurystykę oraz liczenie wspólnych sąsiadów:

```c
static int find_cycle(AdjacencyList *adj_list, int **cycle, int *cycle_len) {
  return (find_cycle_perimeter(adj_list, cycle, cycle_len) == SUCCESS)
              ? SUCCESS
              : find_cycle_backup(adj_list, cycle, cycle_len);
}

static int count_common_neighbors(int u, int v, AdjacencyList *adj) {
  int common = 0;
  Neighbor *nu = adj->adjacency_list[u];
  while (nu != NULL) {
    Neighbor *nv = adj->adjacency_list[v];
    while (nv != NULL) {
      if (nu->node_index == nv->node_index)
        common++;
      nv = nv->next;
    }
    nu = nu->next;
  }
  return common;
}
```

Fragment pokazujący heurystykę obwodu i fallback:

```c
static int find_cycle_perimeter(AdjacencyList *adj_list, int **cycle_out,
                                int *cycle_len_out) {
  int n = adj_list->nodes_count;
  int start_node = 0;
  int min_deg = adj_list->degrees[0];
  for (int i = 1; i < n; i++) {
    if (adj_list->degrees[i] < min_deg) {
      min_deg = adj_list->degrees[i];
      start_node = i;
    }
  }

  int *path = (int *)malloc(n * sizeof(int));
  if (!path)
    return MEMORY_ERROR;
}

static int find_cycle_backup(AdjacencyList *adj_list, int **cycle_out,
                             int *cycle_len_out) {
  int n = adj_list->nodes_count;
  int *cycle = (int *)malloc(4 * sizeof(int));
  if (!cycle)
    return MEMORY_ERROR;

  int max_indices[4] = {0, 1, 2, 3};
  for (int i = 4; i < n; i++) {
    int min_idx = 0;
    for (int j = 1; j < 4; j++) {
      if (adj_list->degrees[max_indices[j]] <
          adj_list->degrees[max_indices[min_idx]]) {
        min_idx = j;
      }
    }
    if (adj_list->degrees[i] > adj_list->degrees[max_indices[min_idx]]) {
      max_indices[min_idx] = i;
    }
  }
```

```
  for (int i = 0; i < 4; i++) {
    cycle[i] = max_indices[i];
  }

  *cycle_out = cycle;
  *cycle_len_out = 4;
  return SUCCESS;
}
```

Fragment pokazujący układ ramy i wierzchołki małych grafów:

```
static void place_cycle_on_square(Graph *graph, int *cycle, int cycle_len,
                                  double size) {
  double perimeter = 4.0 * size;
  double step = perimeter / cycle_len;
  for (int i = 0; i < cycle_len; i++) {
    double pos = i * step;
    double x, y;
    if (pos < size) {
      x = pos;
      y = 0.0;
    } else if (pos < 2.0 * size) {
      x = size;
      y = pos - size;
    } else if (pos < 3.0 * size) {
      x = size - (pos - 2.0 * size);
      y = size;
    } else {
      x = 0.0;
      y = size - (pos - 3.0 * size);
    }
    graph->nodes[cycle[i]].x = x;
    graph->nodes[cycle[i]].y = y;
  }
}

static void handle_graph_below_four_nodes(Graph *graph, double size) {
  double coords[][2] = {{size / 2.0, size / 2.0},
                        {0.0, 0.0},
                        {size, size},
                        {0.0, 0.0},
                        {size, 0.0},
                        {size / 2.0, size}};
  int start = (graph->nodes_count == 1) ? 0 : (graph->nodes_count == 2) ? 1 : 3;
  for (int i = 0; i < graph->nodes_count; i++) {
    graph->nodes[i].x = coords[start + i][0];
    graph->nodes[i].y = coords[start + i][1];
  }
}
```

6.6 Moduł `utils`

Rola:

- definicja kodów statusu,
- mini-biblioteka listy jednokierunkowej (`list_contains`, `list_prepend`, `list_free`).

Znaczenie:

- jeden punkt definicji kodów błędów,

- prosty komponent wspierający odczyt grafu bez zależności zewnętrznych.

Krótki fragment pokazujący operacje na liście jednokierunkowej:

```
int list_contains(List *head, int value) {
  for (List *cur = head; cur != NULL; cur = cur->next)
    if (cur->data == value)
      return 1;
  return 0;
}

int list_prepend(List **head, int value) {
  List *node = (List *)malloc(sizeof(List));
  node->data = value;
  node->next = *head;
  *head = node;
  return SUCCESS;
}
```

Fragment pokazujący sprzątanie listy:

```
void list_free(List *head) {
  while (head != NULL) {
    List *tmp = head->next;
    free(head);
    head = tmp;
  }
}
```

Fragment pokazujący szukanie cyklu po obwodzie i wybór fallbacku:

```
static int find_cycle_perimeter(AdjacencyList *adj_list, int **cycle_out,
                      int *cycle_len_out) {
  int start_node = 0;
  int min_deg = adj_list->degrees[0];
  for (int i = 1; i < adj_list->nodes_count; i++) {
    if (adj_list->degrees[i] < min_deg) {
      min_deg = adj_list->degrees[i];
      start_node = i;
    }
  }

  if (!found)
    break;
  return ALGORITHM_ERROR;
}
```

Fragment pokazujący definicję kodów statusu i nagłówek listy:

```
enum {
  SUCCESS = 0,
  ARGUMENTS_ERROR = 1,
  FILE_ERROR = 2,
  INPUT_FORMAT_ERROR = 3,
  ALGORITHM_ERROR = 4,
  MEMORY_ERROR = 5,
};

typedef struct List {
  int data;
  struct List *next;
} List;
```

7 Szczegółowa analiza funkcji

7.1 `main(int argc, char *argv[])`

Wejście:

- `argc`, `argv` z systemu operacyjnego.

Wyjście:

- kod procesu zgodny z enum statusów.

Założenia:

- funkcja oczekuje dwóch argumentów pozycyjnych i opcjonalnych flag,
- dla opcji wymagających argumentu oczekuje poprawnego tokenu po opcji,
- po poprawnej konfiguracji wywołuje dokładnie jeden algorytm i jeden zapis.

Sytuacje wyjątkowe:

- nieznana opcja,
- brak plików,
- niepoprawne wartości `iterations`, `temperature`, `size`,
- nieznany algorytm lub format.

Uwaga: fragmenty kodu dla tej funkcji znajdują się wyłącznie w sekcji `Szczegółowy opis kodu`.

7.2 `load_graph(char path[], Graph **graph_out)`

Wejście:

- ścieżka do pliku tekstowego.

Wyjście:

- wskaźnik do zaalokowanej struktury `Graph`.

Założenia:

- plik musi być czytelny,
- każdy rekord danych musi mieć 4 pola (`label int int double`),
- komentarze rozpoczynają się od `#`.

Inwarianty po sukcesie:

- `nodes_count >= 0`, `edges_count >= 0`,
- tablica `nodes` jest posortowana rosnąco po `id`,
- każda krawędź ma poprawne indeksy do tablicy `nodes`.

Złożoność:

- skan 1: $O(E \cdot U)$ dla budowy zbioru unikalnych ID na liście (gdzie `U` to liczba unikalnych wierzchołków),
- sortowanie: $O(V \log V)$,
- skan 2: $O(E \log V)$ przez `bsearch`.

Uwaga: fragmenty kodu dla tej funkcji znajdują się wyłącznie w sekcji `Szczegółowy opis kodu`.

7.3 `save_graph_as_text(Graph *graph, char path[])`

Założenia:

- dla każdego wierzchołka zapisuje jeden rekord `id x y`,

- brak nagłówka i metadanych,
- dokładność liczbowa zgodna z `%lf`.

Uwagi:

- format jest najprostszy do debugowania i ręcznej walidacji,
- może być większy rozmiarowo od binarnego przy dużych grafach.

Uwaga: fragmenty kodu dla tej funkcji znajdują się wyłącznie w sekcji `Szczegółowy opis kodu`.

7.4 `save_graph_as_binary(Graph *graph, char path[])`

Założenia:

- każdy rekord ma 20 bajtów,
- dane są zapisywane sekwencyjnie bez separatorów.

Uwagi integracyjne:

- format jest szybki w odczycie, ale mniej odporny na niezgodności platformowe,
- w praktyce warto dodać nagłówek wersji formatu.

Uwaga: fragmenty kodu dla tej funkcji znajdują się wyłącznie w sekcji `Szczegółowy opis kodu`.

7.5 `run_fruchterman(...)`

Warunki wejściowe:

- `graph != NULL`,
- liczba iteracji dodatnia (walidowana wyżej),
- `size > 0`.

Warunki wyjściowe:

- pozycje wierzchołków znajdują się w granicach `[0,size]`,
- tablice pomocnicze są zwolnione.

Uwagi o jakości wyniku:

- dla grafów rzadkich przy niskiej temperaturze możliwe „zamrożenie” układów,
- dla grafów gęstych i dużych wag może występować silne ściskanie klastrów.

Uwaga: fragmenty kodu dla tej funkcji znajdują się wyłącznie w sekcji `Szczegółowy opis kodu`.

7.6 `create_adjacency_list(...)` i `free_adjacency_list(...)`

Założenia tworzenia:

- dla każdej krawędzi nieskierowanej dodawane są dwa wpisy sąsiedztwa,
- `degrees[i]` odzwierciedla stopień i-tego wierzchołka.

Założenia zwalniania:

- wszystkie elementy list sąsiedztwa oraz tablice pomocnicze są zwalniane,
- funkcja toleruje `NULL`.

Uwaga: fragmenty kodu dla tych funkcji znajdują się wyłącznie w sekcji `Szczegółowy opis kodu`.

7.7 `run_tutte(...)`

Warunki wejściowe:

- graf musi być zaalokowany,

- `size > 0`, `max_iterations > 0`.

Warunki wyjściowe:

- punkty ramy pozostają na obwodzie kwadratu,
- wierzchołki wewnętrzne są aktualizowane relaksacyjnie,
- pozycje pozostają w obrębie kwadratu.

Uwagi merytoryczne:

- implementacja nie wyznacza formalnie planarności ani prawdziwej zewnętrznej ściany,
- przy fallbacku 4 najwyższych stopni uzyskany layout może odbiegać od klasycznego embeddingu Tutte.

Uwaga: fragmenty kodu dla tej funkcji znajdują się wyłącznie w sekcji `Szczegółowy opis kodu`.

7.8 Funkcje pomocnicze `utils`

`list_contains`, `list_prepend`, `list_free` tworzą minimalny, celowy zestaw narzędzi.

Silne strony:

- prostota i przewidywalność,
- brak ukrytych efektów ubocznych.

Słabości:

- przy bardzo dużych danych koszt wyszukiwania liniowego w `list_contains` może być istotny.

Uwaga: fragmenty kodu dla funkcji pomocniczych znajdują się wyłącznie w sekcji `Szczegółowy opis kodu`.

8 Opis algorytmiczny

8.1 Fruchterman-Reingold

Model matematyczny:

- optymalna odległość:

$$k = \sqrt{\frac{A}{|V|}}, \quad A = s^2$$

gdzie s oznacza bok obszaru roboczego (`size`).

- siła odpychania:

$$f_{r(d)} = \frac{k^2}{d}$$

- siła przyciągania z wagą:

$$f_{a(d)} = w \cdot \frac{d^2}{k}$$

Złożoność obliczeniowa:

- odpychanie: $O(|V|^2)$ na iteracje,
- przyciąganie: $O(|E|)$ na iteracje,
- całkowicie: $O((|V|^2 + |E|) \cdot I)$.

Właściwości:

- dobrze działa dla szerokiej klasy grafów,
- kosztowne dla dużych grafów,
- podatne na minima lokalne.

8.2 Tutte-like embedding

Idea:

- część wierzchołków jest unieruchomiona na brzegu,
- pozostałe przyjmują średnią ważoną położeń sąsiadów.

Równania relaksacji:

$$x_i = \frac{\sum_{j \in N(i)} w_{ij} x_j}{\sum_{j \in N(i)} w_{ij}}$$

$$y_i = \frac{\sum_{j \in N(i)} w_{ij} y_j}{\sum_{j \in N(i)} w_{ij}}$$

Warunek stopu:

- `max_displacement < epsilon` lub
- przekroczenie limitu `max_iterations`.

Właściwości:

- duża stabilność dla grafów średniej wielkości,

- deterministyczność przy danym grafie
- jakość zależy od trafności wyboru ramy.

9 Zarządzanie pamięcią i odporność

9.1 Strategia alokacji

Stosowane są:

- `malloc` dla obiektów i tablic,
- `calloc` dla tablic wymagających zerowania,
- kontrola null po każdej alokacji.

9.2 Strategia cleanup

W przypadku błędu:

- funkcje zwalniają to, co zostało już zaalokowane,
- zwracany jest adekwatny kod błędu,
- wyższa warstwa (`main`) dokańcza cleanup obiektu `Graph`.

9.3 Potencjalne punkty ryzyka pamięci

- przy bardzo dużych grafach koszt pamięci list sąsiedztwa i tablic tymczasowych może być wysoki,
- brak limitów na liczbę rekordów pliku może prowadzić do wyczerpania zasobów systemowych.

10 Obsługa błędów

10.1 Kategorie błędów

Program obsługuje:

- błędy argumentów i konfiguracji,
- błędy operacji plikowych,
- błędy formatu danych,
- błędy algorytmiczne,
- błędy alokacji pamięci.

10.2 Kody powrotu i znaczenie

- `0` - sukces,
- `1` - błąd argumentów,
- `2` - błąd plikowy,
- `3` - niepoprawny format wejściowy,
- `4` - błąd algorytmu,
- `5` - błąd pamięci.

10.3 Komunikaty diagnostyczne

Najważniejsze komunikaty wyświetlane na `stderr`:

- `Error: Unknown option ...`
- `Error: Input and output files must be specified.`
- `Error: Invalid iteration value`
- `Error: Invalid temperature value`
- `Error: Invalid size value`
- `Error: Cannot open input file: ...`
- `Error: Graph must have at least one node and one edge.`
- `Error: Memory allocation failed`

11 Proces budowania i uruchamiania

11.1 Makefile

Cele:

- `make` - budowa `bin/graph`,
- `make test` - budowa i uruchomienie testów,
- `make clean` - usunięcie `bin`.

Flagi kompilacji:

- `-Wall`
- `-lm`

Uwagi:

- linkowanie z `-lm` zapewnia funkcje matematyczne (`sqrt`, `fmin`),
- testy linkują się z obiektami projektu poza `main.o`.

Aktualna zawartość pliku `Makefile`:

```
CC = gcc
CFLAGS = -Wall -lm
SRC_DIR = src
OBJ_DIR = bin/obj
BIN_DIR = bin
TEST_DIR = tests

TARGET = $(BIN_DIR)/graph
TEST_TARGET = $(BIN_DIR)/test

SRCS = $(wildcard $(SRC_DIR)/*.c)
OBJS = $(patsubst $(SRC_DIR)/%.c,$(OBJ_DIR)/%.o,$(SRCS))

TEST_OBJS_REQUIRED = $(filter-out $(OBJ_DIR)/main.o, $(OBJS))
TEST_SRCS = $(wildcard $(TEST_DIR)/*.c)

all: $(TARGET)

$(TARGET): $(OBJS)
  @mkdir -p $(BIN_DIR)
  $(CC) $(CFLAGS) $(OBJS) -o $(TARGET)

$(OBJ_DIR)/%.o: $(SRC_DIR)/%.c
  @mkdir -p $(OBJ_DIR)
  $(CC) $(CFLAGS) -c $< -o $@

test: $(TEST_TARGET)
  ./$(TEST_TARGET)

$(TEST_TARGET): $(TEST_SRCS) $(TEST_OBJS_REQUIRED)
  @mkdir -p $(BIN_DIR)
  $(CC) $(CFLAGS) $(TEST_SRCS) $(TEST_OBJS_REQUIRED) -o $(TEST_TARGET)

clean:
  rm -rf $(BIN_DIR)

.PHONY: all clean test
```

11.2 Instrukcja uruchomienia

Kompilacja:

```
make
```

Uruchomienie testów:

```
make test
```

Uruchomienie programu:

```
./bin/graph [opcje] input.txt output.txt
```

12 Testowanie i weryfikacja

12.1 Zakres testów jednostkowych

Zestaw testów pokrywa:

- odczyt grafu i mapowanie ID (`test_graph.c`),
- obsługę komentarzy i błędnego formatu,
- tworzenie list sasiedztwa i stopnie (`test_adjacency_list.c`),
- zachowanie `run_fruchterman` dla grafu pustego i małego,
- zachowanie `run_tutte` dla grafu małego i grafu pełnego.

Przykładowe fragmenty testów:

Uruchamianie całego zestawu (`tests/main_test.c`):

```
int main()
{
  fprintf(stdout, "Running tests...\n");

  mute_stderr();

  test_get_node_index();
  test_load_graph_success();
  test_load_graph_file_error();
  test_load_graph_format_error();
  test_load_graph_with_comments();
  test_load_graph_with_inline_comments();
  test_load_graph_with_mixed_comments();

  test_adjacency_list_creation();
  test_adjacency_list_degrees();
  test_adjacency_list_empty_graph();

  test_tutte_small_graph();
  test_tutte_complete_graph();

  test_fruchterman_empty_graph();
  test_fruchterman_small_graph();

  fprintf(stdout, "All tests passed!\n");

  return 0;
}
```

Walidacja odczytu grafu i mapowania (`tests/test_graph.c`):

```
void test_load_graph_success() {
  printf("Testing load_graph success... ");
  const char *path = "test_ok.txt";
  create_dummy_file(path, "# Komentarz\n"
                          "L1 1 2 1.5\n"
                          "L2 2 3 0.5\n");

  Graph *g = NULL;
  int status = load_graph((char *)path, &g);

  assert(status == 0);
  assert(g != NULL);
```

```
  assert(g->nodes_count == 3);
  assert(g->edges_count == 2);

  int idx1 = get_node_index(g, 1);
  int idx2 = get_node_index(g, 2);
  int first = g->edges[0].first_node_index;
  int second = g->edges[0].second_node_index;

  assert((first == idx1 && second == idx2) ||
         (first == idx2 && second == idx1));
  assert(g->edges[0].weight == 1.5);

  free_graph(g);
  remove(path);
  printf("PASSED\n");
}
```

Sprawdzanie listy sąsiedztwa i stopni (`tests/test_adjacency_list.c`):

```
void test_adjacency_list_creation() {
  printf("Testing adjacency list creation... ");

  Graph *g = (Graph *)malloc(sizeof(Graph));
  g->nodes_count = 3;
  g->edges_count = 2;
  g->nodes = (Node *)malloc(3 * sizeof(Node));
  g->edges = (Edge *)malloc(2 * sizeof(Edge));

  g->edges[0].first_node_index = 0;
  g->edges[0].second_node_index = 1;
  g->edges[0].weight = 1.5;

  g->edges[1].first_node_index = 1;
  g->edges[1].second_node_index = 2;
  g->edges[1].weight = 2.0;

  AdjacencyList *adj_list;
  int result = create_adjacency_list(g, &adj_list);

  assert(result == SUCCESS);
  assert(adj_list->degrees[0] == 1);
  assert(adj_list->degrees[1] == 2);
  assert(adj_list->degrees[2] == 1);

  free_adjacency_list(adj_list);
  free_graph(g);
  printf("PASSED\n");
}
```

Testy algorytmów layoutu (`tests/test_tutte.c`, `tests/test_fruchterman.c`):

```
void test_tutte_small_graph() {
  Graph *g = (Graph *)malloc(sizeof(Graph));
  g->nodes_count = 2;
  g->edges_count = 1;
  g->nodes = (Node *)malloc(2 * sizeof(Node));
  g->edges = (Edge *)malloc(1 * sizeof(Edge));

  g->edges[0].first_node_index = 0;
  g->edges[0].second_node_index = 1;
```

```
  g->edges[0].weight = 1.0;

  int result = run_tutte(g, 1000.0, 100);
  assert(result == SUCCESS);
  assert(g->nodes[0].x == 0.0);
  assert(g->nodes[1].x == 1000.0);

  free_graph(g);
}

void test_fruchterman_empty_graph()
{
  Graph *graph = (Graph *)malloc(sizeof(Graph));
  graph->nodes = NULL;
  graph->nodes_count = 0;
  graph->edges = NULL;
  graph->edges_count = 0;

  int result = run_fruchterman(graph, 10.0, 1000, 1000.0);
  assert(result == SUCCESS);
  free(graph);
}
```

12.2 Wynik aktualnego uruchomienia

W bieżącej wersji projektu testy przechodzą poprawnie (`All tests passed!`).

12.3 Ograniczenia algorytmiczne

- Fruchterman-Reingold ma koszt kwadratowy względem liczby wierzchołków.
- Jakość layoutu zależy od parametrów (`iterations`, `temperature`, `size`).
- Tutte-like embedding zależy od heurystycznego wyboru ramy.

12.4 Ograniczenia implementacyjne

- Brak parsera z „twardą” diagnostyką numerów linii (kod `line_number` jest utrzymywany, ale nie jest eksponowany w komunikatach).
- Brak wsparcia dla danych binarnych na wejściu (tylko tekst).
- Brak opcji wymuszenia ziarna RNG w Fruchterman (utrudniona reprodukowalność).

12.5 Ryzyka eksploatacyjne

- Dla bardzo dużych grafów czas i pamięć mogą być nieakceptowalne bez optymalizacji.
- Niepoprawne dane (np. semantycznie niespójny graf) mogą prowadzić do layoutu o niskiej jakości bez jednoznacznego błędu.

13 Podsumowanie

Projekt C realizuje wymagany trzon funkcjonalny: odczyt danych grafu, dwa algorytmy wyznaczania layoutu, eksport wyników i obsługę uruchomienia z linii poleceń. Implementacja jest modularna, czytelna i testowalna. Niniejsza dokumentacja końcowa łączy perspektywę użytkową i implementacyjną, opisuje kod szczegółowo oraz wskazuje rzeczywiste granice systemu.

W tej postaci projekt jest gotowy do przekazania dalej jako silnik obliczeniowy i baza dla kolejnego etapu (warstwa wizualna i interakcyjna).

14 Załącznik A: Skrócona referencja API (C)

14.1 graph.h

- `int load_graph(char path[], Graph **graph_out)`
- `int save_graph_as_binary(Graph *graph, char path[])`
- `int save_graph_as_text(Graph *graph, char path[])`
- `int free_graph(Graph *graph)`
- `int get_node_index(Graph *graph, int node_id)`

14.2 fruchterman.h

- `int run_fruchterman(Graph *graph, double initial_temperature, int max_iterations, double size)`

14.3 tutte.h

- `int run_tutte(Graph *graph, double size, int max_iterations)`

14.4 adjacency_list.h

- `int create_adjacency_list(Graph *graph, AdjacencyList **adj_list_out)`
- `void free_adjacency_list(AdjacencyList *adj_list)`

14.5 utils.h

- `int list_contains(List *head, int value)`
- `int list_prepend(List **head, int value)`
- `void list_free(List *head)`

15 Załącznik B: Przykładowe dane

15.1 Przykładowy plik wejściowy

```
# Prostokąt z przekątną
AB 1 2 1.0
BC 2 3 1.0
CD 3 4 1.0
DA 4 1 1.0
AC 1 3 1.2
```

15.2 Przykładowe uruchomienie

```
./bin/graph -a tutte -i 1200 -s 1000 -f text input.txt output.txt
```

15.3 Przykładowy wynik tekstowy

```
1 0.000000 0.000000
2 1000.000000 0.000000
3 1000.000000 1000.000000
4 0.000000 1000.000000
```